

The Semester Project

File Systems-III

This article, which is part of the series on Linux device drivers, completes the basic simulation of a file system in user space.

Part—20

Till now, Shweta had implemented four basic functionalities of the simulated file system (Simula FS) browser—*quit* (browser), *list* (files), *create* (an empty file), and *remove* (a file). Here, she adds three little advanced functionalities, to get the feel of a complete basic file system:

- Changing the permissions of a file
- Reading from a file
- Writing into a file

Here's a sneak peek into her thinking process as she came up with her implementation. For the various command implementations, two very common requirements keep occurring quite often:

- Getting the index of the entry for a particular filename
- Updating the entry for a given filename

Hence, these two requirements, captured by the functions *sfs_lookup()* and *sfs_update()*:

```
int sfs_lookup(int sfs_handle, char *fn, sfs_file_entry_t *fe) {
    int i;

    lseek(sfs_handle, sb.entry_table_block_start * sb.block_size,
    SEEK_SET);

    for (i = 0; i < sb.entry_count; i++) {
        read(sfs_handle, fe, sizeof(sfs_file_entry_t));
        if (!fe->name[0]) continue;
        if (strcmp(fe->name, fn) == 0) return i;
    }

    return -1;
}
```

```
void sfs_update(int sfs_handle, char *fn, int *size, int update_
```

```
ts, int *perms) {
    int i;
    sfs_file_entry_t fe;

    if ((i = sfs_lookup(sfs_handle, fn, &fe)) == -1) {
        printf("File %s doesn't exist\n", fn);
        return;
    }

    if (size) fe.size = *size;
    if (update_ts) fe.timestamp = time(NULL);
    if (perms && (*perms <= 07)) fe.perms = *perms;
    lseek(sfs_handle, sb.entry_table_block_start * sb.block_size
+ i * sb.entry_size, SEEK_SET);
    write(sfs_handle, &fe, sizeof(sfs_file_entry_t));
}
```

The *sfs_lookup()* function traverses through all the entries (skipping the invalid, i.e., empty filename entries), till it finds the matching filename, and then returns the index and the entry of the matched entry in the function's third parameter. It returns -1 in case no match is found.

Now, *sfs_update()* uses *sfs_lookup()* to get the entry and its index for the given filename. Then, it updates it back into the filesystem with: 1. size, if passed (i.e., non-NULL); 2. current timestamp if *update_ts* is set; and 3. permissions, if passed (i.e., non-NULL).

Changing file permissions

In the Simula FS, file permissions are basically a combination of 'rwx', for the user only, stored as an integer, with Linux-like notation of 4 for 'r', 2 for 'w', and 1 for 'x'. To change